

COMPSYS302: Python

4. Module

Ho Seok AHN

hs.ahn@auckland.ac.nz

4.1 About Module

□ Modules

- A typical Python program is made up of several source files. Each source file corresponds to a **module**, which packages program code and data for reuse.
- **Modules** are normally independent of each other so that other programs can reuse the specific modules they need.
- A module explicitly establishes dependencies up to another module by using **import** or **from** statements.
- A module is a Python object with arbitrarily named attributes that you can bind and reference.
- In Python, **modules are objects** and are handled like other objects. Thus, you can pass a module as an argument in a call to a function.

4.2 Use of Module

□ Python built-ins

- The main purpose of using module is reusing source code.
- If you do not use module, you should make all of them by yourself.
- Python offers several built-in objects, which are “**already existing functions**”.
- All built-in objects are attributes of a preloaded module named ‘__builtin__’.

▪ Ex) 4_01_Builtins.py

```
print(globals())      # for checking current attributes
print(dir('__builtin__')) # for checking current attributes in '__builtin__'
```

(result)

```
{'__builtins__': <module '__builtin__' (built-in)>, '__name__': '__main__', '__file__': 'COMPSYS302\ \ 4_01_Builtins.py',
 '__doc__': '\ n4_01_Builtins.py\ n', '__package__': None}
['__add__', '__class__', '__contains__', '__delattr__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__',
 '__getitem__', '__getnewargs__', '__getslice__', '__gt__', '__hash__', '__init__', '__le__', '__len__', '__lt__', '__mod__',
 '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__',
 '__str__', '__subclasshook__', '_formatter_field_name_split', '_formatter_parser', 'capitalize', 'center', 'count', 'decode',
 'encode', 'endswith', 'expandtabs', 'find', 'format', 'index', 'isalnum', 'isalpha', 'isdigit', 'islower', 'isspace', 'istitle', 'isupper',
 'join', 'ljust', 'lower', 'lstrip', 'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith',
 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

4.2 Use of Module

□ Use

- Python offers various modules as well.
- For example,
 - String, date, time, **math**, fractions, decimal, random, file, sqlite3, ftp, http, os, sys, threading, unittest, xml, email, etc.
- How to use?
 - Ex) 4_02_MathModule.py

```
import math
```

```
print (math.pow(2, 10))
```

```
print (math.log(100))
```

```
print (math.pi)
```

(result)

1024.0

4.60517018599

3.14159265359

4.2 Use of Module

□ The **import** Statement

- You can use any Python source file as a module by executing an **import** statement in some other code.
- **import** has the following syntax:

`import modulename [as variblename][,...]`

- The **import** keyword is followed by one or more module specifiers, separated by commas.
- When 'as variblename' is part of an **import** statement, Python binds the variable named 'variblename' to the module object, but the module name that Python looks for is 'modulename'.

- Ex) 4_03_ModuleImport.py

```
import DefineFunction as myModule  
print (myModule.Times(10, 20))
```

(result)

200

4.2 Use of Module

□ The from Statement

- Python's **from** statement lets you import specific attributes from a module into the current namespace.
- **from** has two syntax:

`from modulename import *`

`from modulename import attributename [as variblename][,...]`

- A **from** statement specifies a module name, followed by one or more attribute specifiers, separated by commas.
- When 'as variblename' is part of a **from** statement, Python binds the variable named 'variblename' to the attribute, but the module attribute from which the variable gets its value is 'attributename'.

- Ex) 4_04_ModuleFrom.py

```
from DefineFunction import *  
print (Times(10, 20))
```

(result)

200

```
from DefineFunction import Times as myTime  
print (myTime(10, 20))
```

(result)

200